

Projektarbeit I

Studiengang Wirtschaftsinformatik – Software Engineering

Fakultät für Wirtschaft Duale Hochschule Baden-Württemberg, Mannheim

Feature Entwicklung: Integration der Viskositätsindex-Berechnung in die
Engineering App mit Angular

Name des Verfassers:	Duong, Thi Quynh Nhi
Matrikelnummer:	6889454
E-Mail-Adresse:	s241418@student.dhbw-mannheim.de
Anschrift:	Adam-Klein-Straße 83a, 90429 Nürnberg
Kurs:	WWI24SEA
Studiengangsleiter:	Professor Dr. Hans-Henning Pagnia
Betrieblicher Betreuer:	Moritz Vogel
Wissenschaftlicher Betreuer:	Phillip Schreiner
Finaler Abgabetermin:	25.08.2025

Inhaltsverzeichnis

1. Einleitung	4
1.1 Motivation und Hintergrund	4
1.2 Zielsetzung der Projektarbeit	5
1.3 Aufbau der Arbeit	5
2. Theorieanteil: Einführung in Angular-Entwicklung	5
2.1 Moderne Webentwicklung	5
2.2 Single-Page Applications (SPA)	6
2.3 Responsive Design.....	6
2.4 Was ist Angular?	7
2.5 Einführung in TypeScript.....	8
2.6 Warum wurde das Projekt mit Angular umgesetzt?	8
2.7 Komponentenbasierte Architektur.....	9
2.8 Data binding	9
2.8.1 Grundlage der Datenbindung (Data Binding).....	10
2.8.2 One-way Data Bindings	10
2.8.3 Event-Binding.....	10
2.8.4 Two-way Data Bindings	11
2.9 Formulare	11
2.9.1 Template-driven Forms.....	12
2.9.2 Reactive Forms	12
2.9.3 Gegenüberstellung von Template-driven und Reactive Forms.....	14
2.10 Validierung von Eingaben in Formularen	14
2.10.1 Globale CSS-Validierung:	14
2.10.2 Validierung in Template-driven Forms	14
2.10.3 Validierung in Reactive Forms	15
2.10.4 Benutzerdefinierte Validierung	15
2.11 Einführung in Angular Material	16
2.11.1 Material Design	16
2.11.2 Angular Material	17
3. Grundlagen des Viskositätsindex	17
4. Praxisanteil: Entwicklung des Features	17
4.1 Validierung der Benutzereingaben	18
4.2 Formularstruktur und Eingabefelder	18
4.3 Berechnung des Viskositätsindex	19
4.4 Integration in die Benutzeroberfläche	20

5. Reflexion und Ausblick.....	21
5.1 Bewertung der Umsetzung	21
5.2 Herausforderungen und Lösungsansätze	21
5.3 Potenzial für Weiterentwicklung.....	21
6. Fazit	22
7. Literaturverzeichnis	22
8. Hilfsmittelangabe	23
9. Selbstständigkeitserklärung.....	25
10. Sperrvermerk	25

1. Einleitung

Die präzise Auswahl und Anwendung von Schmierstoffen ist entscheidend für die Leistungsfähigkeit und Langlebigkeit technischer Systeme. Digitale Tools, die Anwender bei dieser Aufgabe unterstützen, gewinnen daher zunehmend an Bedeutung. Im Rahmen des webbasierten Engineering App-Ökosystems von Schaeffler steht Anwendern eine Vielzahl spezialisierter Tools zur Verfügung, darunter auch die Grease App, die ein zentrales Hilfsmittel zur Auslegung und Anwendung von Schmierstoffen darstellt. Mit ihrer benutzerfreundlichen Oberfläche ermöglicht sie die einfache Bestimmung der passenden Arcanol-Fettsorte, der Fettgebrauchsdauer, des Nachschmierintervalls sowie der Schmierstoffmenge für die Erst- und Nachschmierung. Die App basiert auf dem bewährten Berechnungsprogramm Bearinx und steht sowohl als Webanwendung als auch als mobile App zur Verfügung. So liefert sie verlässliche Ergebnisse, direkt dort, wo sie benötigt werden, etwa in Werkstätten oder im Außeneinsatz.

Im Zuge der Weiterentwicklung dieses digitalen Angebots soll die Grease App nun um ein weiteres praxisrelevantes Feature ergänzt werden: einen integrierten Viskositätsindex-Rechner. Der Viskositätsindex (VI) ist ein bedeutender Kennwert zur Beurteilung der Temperaturstabilität von Schmierstoffen. Seine zuverlässige Berechnung ermöglicht es Anwendern, fundierte Entscheidungen bei der Produktauswahl zu treffen.

Um diesen Bedarf zu decken, wird ein neues Frontend-Feature zur automatisierten Berechnung des VI entwickelt. Es erlaubt die Eingabe der kinematischen Viskositätswerte bei 40°C und 100°C und liefert daraus direkt den berechneten VI-Wert. Dieses Feature wird als eigenständige Komponente in die Grease App integriert, um eine durchgängige und medienübergreifende Nutzererfahrung zu schaffen.

1.1 Motivation und Hintergrund

Die Digitalisierung technischer Hilfsmittel zur Instandhaltung und Auslegung von Produkten ist ein wesentlicher Faktor für Effizienz und Prozesssicherheit. Mit der Grease App bietet Schaeffler bereits ein etabliertes digitales Werkzeug, das Anwender bei der Auswahl und Anwendung von Schmierstoffen unterstützt, unter anderem durch die Berechnung von Nachschmierintervallen, Fettgebrauchsdauer und Schmierstoffmengen.

Ein häufig genannter Anwendungswunsch aus dem Nutzerkreis betrifft die Berechnung des Viskositätsindex (VI), ein zentraler Kennwert zur Beurteilung der Temperaturstabilität von Schmierstoffen. Dieser Wert ist insbesondere dann entscheidend, wenn Lager unter stark schwankenden oder extremen Temperaturbedingungen betrieben werden.

Die Anforderung wurde durch die folgende User Story prägnant formuliert:

„As a User I want to calculate the Viscosity Index by entering the viscosity at 40° and 100° so I don't have to use competitor applications.“

Diese Aussage verdeutlicht den Wunsch nach einer integrierten, benutzerfreundlichen Lösung im bestehenden Schaeffler-Ökosystem, ohne auf externe Tools oder Konkurrenzprodukte zurückgreifen zu müssen. Die Erweiterung der Grease App um einen VI-Rechner schließt diese funktionale Lücke und stärkt gleichzeitig die Attraktivität und Nutzungsbreite der Anwendung.

Aus technischer Sicht bietet die Implementierung die Möglichkeit, die bestehende Angular-Architektur modular zu erweitern und das neue Feature als Standalone-Komponente bereitzustellen. UX-seitig liegt

der Fokus auf einer intuitiven Benutzerführung mit klaren Eingabefeldern und direktem Ergebnisfeedback, um auch in mobilen Einsatzszenarien eine hohe Bedienbarkeit zu gewährleisten.

1.2 Zielsetzung der Projektarbeit

Ziel dieser Projektarbeit ist die Konzeption, Entwicklung und Integration eines digitalen Features zur Berechnung des Viskositätsindex (VI) innerhalb der webbasierten Schaeffler Grease App. Dieses Feature soll es Anwendern ermöglichen, den VI-Wert eines Schmierstoffs schnell und einfach durch die Eingabe der kinematischen Viskositäten bei 40°C und 100°C zu berechnen.

Das Feature soll:

- Nahtlos in die bestehende Grease App integriert werden, ohne bestehende Funktionen zu stören,
- Intuitiv bedienbar sein, sowohl auf mobilen als auch auf Desktop-Geräten,
- Auf der Grundlage standardisierter Berechnungsformeln präzise Ergebnisse liefern,
- Modular und wiederverwendbar aufgebaut sein.

Durch dieses Feature wird die Funktionalität der Grease App sinnvoll erweitert und ein wichtiger Mehrwert für Anwender geschaffen, die auf zuverlässige Viskositätsdaten zur Produktauswahl angewiesen sind.

1.3 Aufbau der Arbeit

Die Projektarbeit gliedert sich in sechs Hauptkapitel. Nach der Einleitung, in der Motivation, Zielsetzung und Aufbau der Arbeit erläutert werden, folgt im zweiten Kapitel der theoretische Teil zur Angular-Entwicklung. Dort werden Grundlagen moderner Webentwicklung, die Architektur von Angular, die Verwendung von TypeScript sowie die Formularverarbeitung und Validierung behandelt.

Kapitel 3 widmet sich den Grundlagen des Viskositätsindex und erklärt dessen Bedeutung sowie die Berechnung nach der ASTM D2270-Norm. Im vierten Kapitel wird der Praxisanteil der Arbeit vorgestellt. Hier liegt der Fokus auf der konkreten Umsetzung des VI-Rechners mit Angular, einschließlich Validierung, Berechnungslogik und Integration in die Benutzeroberfläche.

Kapitel 5 enthält die Reflexion und den Ausblick. Es werden die Umsetzung bewertet, Herausforderungen beschrieben und mögliche Weiterentwicklungen aufgezeigt. Abschließend fasst Kapitel 6 die Ergebnisse zusammen und schließt die Arbeit mit einem Fazit ab.

2. Theorieanteil: Einführung in Angular-Entwicklung

2.1 Moderne Webentwicklung

Die Webentwicklung hat sich in den letzten Jahren grundlegend verändert. Während früher einfache Webseiten mit statischem Inhalt im Vordergrund standen, erwarten Nutzer*innen heute interaktive, performante und geräteübergreifend optimierte Anwendungen. Diese Entwicklung wurde durch neue Technologien, Frameworks und Architekturen vorangetrieben, die es ermöglichen, komplexe Anwendungen direkt im Browser schnell, dynamisch und benutzerfreundlich auszuführen.

Ein zentrales Merkmal dieser modernen Webentwicklung ist der Wandel hin zu Single Page Applications (SPAs), bei denen Inhalte dynamisch nachgeladen werden, ohne dass die gesamte Seite neu geladen werden muss. Frameworks wie Angular haben diesen Paradigmenwechsel maßgeblich geprägt und bieten leistungsstarke Werkzeuge, um skalierbare und wartbare Webanwendungen zu entwickeln.

2.2 Single-Page Applications (SPA)

In der Anfangsphase der Webentwicklung waren ausschließlich statische Webseiten verbreitet. Dabei handelt es sich um HTML-Dateien, die auf einem Server gespeichert sind und bei einer Anfrage direkt an den Browser gesendet werden. Der Server übernimmt dabei im Grunde lediglich die Rolle eines Dateiservers, der HTML-Dokumente ausliefert. Der Browser wiederum ist dafür zuständig, den empfangenen HTML-Code darzustellen. Eine Interaktion mit dem Browser war lediglich über Hyperlinks möglich (also über das HTML-Element `<a>`), über die man auf eine andere Seite des Servers zugreifen konnte.¹

Mit dem Aufkommen dynamischer Webseiten veränderte sich die Art und Weise der Webseitenerstellung grundlegend. Im Gegensatz zu statischen Seiten wird bei einer Anfrage durch den Browser ein serverseitiges Programm ausgeführt, das die benötigte HTML-Struktur im Arbeitsspeicher generiert und diese anschließend an den Browser übermittelt. Die generierte HTML-Datei wird dabei in der Regel nicht dauerhaft auf dem Datenträger abgelegt. Allerdings kann der Server diese temporär in einem Cache speichern, um spätere Zugriffe zu beschleunigen, sogenanntes Output-Caching. Ein wesentlicher Vorteil dynamischer Seiten liegt in ihrer Flexibilität. Das heißt, dass durch den Zugriff auf Datenbanken mit demselben Programmcode tausende unterschiedliche Seiteninhalte erzeugt werden können. Dieses Prinzip findet insbesondere bei kommerziellen Webanwendungen wie beispielsweise die nächste Seite erneut vom Server generiert und vollständig an den Browser neu gerendert, was die Benutzererfahrung beeinträchtigen kann.²

Im Jahr 1995 entwickelte Brendan Eich die Programmiersprache JavaScript, die heute als ECMAScript standardisiert wird, um einfache Interaktionen direkt im Browser zu ermöglichen. Dadurch konnten Webanwendungen erstmals dynamisch auf Benutzeraktionen reagieren, indem nur ausgewählte Teile der Webseite nachgeladen wurden, anstatt die gesamte Seite neu zu laden. Ein frühes Beispiel für diese Technik ist die Outlook Web Application von Microsoft, welche im Erscheinungsbild und Funktionsumfang stark an die gleichnamige Desktop-Anwendung erinnert. Google's Gmail stellt ein weiteres prominentes Beispiel dar. Beide Anwendungen gelten als sogenannte Single-Page Applications (SPAs). SPAs zeichnen sich dadurch aus, dass nur bestimmte Abschnitte der Webseite während der Laufzeit basierend auf Benutzerinteraktionen ausgetauscht werden. Wird etwa eine E-Mail ausgewählt, ersetzt die entsprechende Ansicht den zentralen Inhaltsbereich der Seite. Bei einem Wechsel zum Posteingang wird dieser Bereich erneut aktualisiert und zeigt die Liste der vorhandenen Nachrichten.³

2.3 Responsive Design

Die erste Website ging im Jahr 1991 online, die ein Meilenstein ist, der den Beginn einer neuen Ära markierte. Seitdem hat sich die Webentwicklung und das Design rasant weiterentwickelt und ist zu einem festen Bestandteil der digitalen Welt geworden. Mit dem Aufkommen von Mobiltelefonen in den 2000er-Jahren und der Einführung des ersten iPhones im Jahr 2007 änderte sich die Art und Weise, wie Menschen auf das Internet zugreifen. Die damaligen Websites waren jedoch ausschließlich für Desktopgeräte konzipiert. Wenn solche Seiten auf einem mobilen Gerät geöffnet wurden, führte das oft zu einer mangelhaften Benutzererfahrung bspw. Text sind schwer lesbar, Navigationselemente zu klein und das Layout nicht für kleine Bildschirme optimiert.⁴

¹ Peter Himschoot, Microsoft Blazor, Building Web Applications in .NET6 and Beyond, Kapitel 9, S.351.

² Peter Himschoot, Microsoft Blazor, Building Web Applications in .NET6 and Beyond, Kapitel 9, S.351.

³ Peter Himschoot, Microsoft Blazor, Building Web Applications in .NET6 and Beyond, Kapitel 9, S.351, 352.

⁴ Amit Diwan, Ultimate Bootstrap for Responsive Web Design, Kapitel 1, Seite 22.

Responsive Webdesign bedeutet, dass eine Website so zu gestalten, dass sie auf allen Geräten, Smartphone, Tablet oder Laptop, einheitlich, zugänglich und zuverlässig dargestellt wird. In der heutigen Zeit greifen Nutzer über eine Vielzahl von Bildschirmgrößen auf Websites zu und die Gestaltung muss sich flexibel an diese Gegebenheiten anpassen.⁵

Responsivität bedeutet, dass sich die Layout-Struktur automatisch verändert, je nach verwendetem Gerät. Wird eine Website bspw. auf einem Tablet geöffnet, sollten die einzelnen Abschnitte so angeordnet sein, dass sie übersichtlich und gut lesbar sind. Auf einem Smartphone hingegen sollten die Inhalte vollständig gestapelt werden, um eine einfache Navigation und ein klares Verständnis des Layouts zu ermöglichen.⁶

Das responsive Design sorgt dafür, dass sich die Website automatisch verkleinert, vergrößert oder umstrukturiert, um sich optimal an die jeweilige Bildschirmgröße anzupassen. Ziel ist es, dem Nutzer auf jedem Gerät eine optimale Benutzererfahrung zu bieten, unabhängig davon, ob er einen Desktop-PC, ein Tablet oder ein Mobiltelefon verwendet.⁷

2.4 Was ist Angular?

In den letzten Jahren hat sich die Webentwicklung rasant weiterentwickelt. Zahlreiche Frameworks, Bibliotheken und Tools sind entstanden, die es Entwickler*innen ermöglichen, leistungsstarke und moderne Webanwendungen zu erstellen. Angular nimmt eine Schlüsselrolle unter den Webframeworks ein, da es konsequent auf Effizienz, moderne Technologien und eine angenehme Entwicklererfahrung ausgerichtet ist.⁸

Angular ist ein Web-Framework, das in der Programmiersprache TypeScript geschrieben wurde. Es umfasst eine Kommandozeilenschnittstelle (CLI), einen Sprachdienst, ein Debugging-Tool sowie eine umfangreiche Sammlung offizieller Bibliotheken.⁹

Angular ermöglicht die Entwicklung skalierbarer Webanwendungen mithilfe von TypeScript, eine strikt typisierten Erweiterung von JavaScript, die in dem nächsten Kapitel behandelt wird.

Google veröffentlichte 2012 die erste Version von Angular unter dem Namen AngularJS. Dieses Framework basiert auf JavaScript und Anwendungen, die damit entwickelt wurden, waren ebenfalls in JavaScript geschrieben. Im Jahr 2016 entschied sich das Angular-Team zu einem grundlegenden Wandel von AngularJS. In Zusammenarbeit mit dem TypeScript-Team von Microsoft wurde die Programmiersprache TypeScript in das Framework integriert. Die darauf folgende Version, Angular 2.0, wurde vollständig in TypeScript geschrieben und unter dem neuen Namen „Angular“ veröffentlicht, inklusive neuem Logo, das sich deutlich von dem von AngularJS unterschied.¹⁰

Im Jahr 2022 leitete Angular eine neue Phase kontinuierlicher Weiterentwicklung ein, bekannt als die Angular Renaissance. In dieser Zeit gewann das Framework erneut an Bedeutung in der Webentwicklung, indem es zentrale Innovationen einführte, die sowohl die Developer Experience (DX) als auch die Anwendungsperformance deutlich verbesserten. Zu den wichtigsten Neuerungen zählten:¹¹

⁵ Amit Diwan, Ultimate Bootstrap for Responsive Web Design, Kapitel 1, Seite 23.

⁶ Amit Diwan, Ultimate Bootstrap for Responsive Web Design, Kapitel 1, Seite 22.

⁷ Amit Diwan, Ultimate Bootstrap for Responsive Web Design, Kapitel 1, Seite 22.

⁸ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 1, Seite 1.

⁹ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 1, Seite 4.

¹⁰ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 1, Seite 4.

¹¹ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 1, Seite 5.

- Ein moderner und vereinfachter Ansatz zur Erstellung von Angular-Anwendungen.
- Verbesserte Reaktivitätsmuster zur effizienten Verwaltung des Anwendungszustands.
- Die Integration von Server-Side Rendering (SSR), um die Performance weiter zu steigern.

Die Veröffentlichung von Angular 17 stellte einen wichtigen Schritt in der Weiterentwicklung des Frameworks dar. Durch das Rebranding mit neuem Logo und modernen Farben brachte das Angular-Team die tiefgreifenden Veränderungen der letzten Jahre zum Ausdruck und unterstrich die Ausrichtung auf zukünftige Innovationen.¹²

2.5 Einführung in TypeScript

Mehr als 20 Jahre nach seiner Einführung gehört JavaScript zu den am weitesten verbreiteten plattformübergreifenden Programmiersprachen weltweit. Ursprünglich als einfache Skriptsprache zur Ergänzung von Webseiten entwickelt, hat sich JavaScript zu einer bevorzugten Sprache für Frontend- und Backend-Anwendungen jeder Größenordnung entwickelt.¹³

Mit dem Wachstum von Umfang und Komplexität moderner JavaScript-Anwendungen ist jedoch ein Problem deutlich geworden: Die Sprache selbst bietet nur begrenzte Möglichkeiten, Beziehungen zwischen verschiedenen Codeeinheiten klar auszudrücken. In Kombination mit den teils ungewöhnlichen Laufzeiteigenschaften von JavaScript führt dies dazu, dass die Entwicklung größerer Anwendungen schwer zu kontrollieren ist.¹⁴

Die häufigsten Fehlerarten in JavaScript sind sogenannte Typfehler, also Situationen, in denen ein bestimmter Wert verwendet wird, obwohl ein anderer erwartet wurde. Solche Fehler entstehen oft durch Tippfehler, Missverständnisse bei der Nutzung von Bibliotheken, falsche Annahmen über das Laufzeitverhalten oder andere Ursachen.¹⁵

TypeScript wurde entwickelt, um diesem Problem entgegenzuwirken. Es handelt sich um einen statischen Typprüfer für JavaScript, das heißt, dass TypeScript den Code bereits vor der Ausführung überprüft und sicherstellt, dass die verwendeten Typen korrekt sind.¹⁶

2.6 Warum wurde das Projekt mit Angular umgesetzt?

Für die Umsetzung des Projekts wurde Angular als Framework gewählt, da es eine moderne, stabile und vielseitige Grundlage für Webanwendungen bietet. Besonders überzeugend sind die plattformübergreifenden Einsatzmöglichkeiten, etwa durch Angular SSR für serverseitiges Rendering oder die Integration mit Ionic zur Entwicklung mobiler Anwendungen. Die Angular CLI sowie Angular DevTools unterstützen eine effiziente Entwicklung durch automatisiertes Scaffolding, Testing und Debugging. Der Einstieg in Angular wird durch eine klare Struktur, umfassende Dokumentation und mitgelieferte Kernbibliotheken wie dem HTTP-Client, dem Router und dem Formularmodul erleichtert. Darüber hinaus profitiert Angular von einer aktiven Entwicklergemeinschaft und wird weltweit in zahlreichen professionellen Projekten, unter anderem bei Google, erfolgreich eingesetzt.¹⁷

¹² Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 1, Seite 5.

¹³ <https://www.typescriptlang.org>, 04.08.2025.

¹⁴ <https://www.typescriptlang.org>, 04.08.2025.

¹⁵ <https://www.typescriptlang.org>, 04.08.2025.

¹⁶ <https://www.typescriptlang.org>, 04.08.2025.

¹⁷ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 1, Seite 5,6.

2.7 Komponentenbasierte Architektur

Komponenten sind spezielle Direktiven, die über eigene Templates verfügen und somit nicht auf extern bereitgestellte Inhalte angewiesen sind. Sie bilden die Grundlage jeder Angular-Anwendung und ermöglichen eine klare Trennung von Struktur, Logik und Darstellung. Wie normale Direktiven besitzen Komponenten ein Host-Element, können Input- und Output-Properties definieren und haben Zugriff auf alle Funktionen, die auch für Direktiven gelten. Der entscheidende Unterschied liegt jedoch darin, dass Komponenten ihre eigene Benutzeroberfläche über ein Template definieren, das direkt mit der Komponente verknüpft ist.¹⁸ Dadurch lassen sich wiederverwendbare, klar strukturierte UI-Bausteine erstellen, die sowohl Daten empfangen als auch Ereignisse auslösen können. Komponenten fördern die Modularität und Wartbarkeit von Anwendungen sind ein zentrales Element in der Architektur in Angular.

Direktiven können leistungsstarke Aufgaben übernehmen, haben jedoch nur begrenzten Einblick in die Elemente, auf die sie angewendet werden. Sie sind besonders dann hilfreich, wenn sie als generische Werkzeuge eingesetzt werden, wie z.B. die `ngModel`-Direktive. Diese kann auf beliebige Datenmodell-Eigenschaften und Formularelemente angewendet werden, unabhängig davon, welche Daten verarbeitet oder welche Elemente verwendet werden.¹⁹

Im Gegensatz zu Direktiven sind Komponenten eng mit dem Inhalt ihrer Templates verknüpft. Sie stellen die Daten und Logik bereit, die durch Datenbindungen in den HTML-Elementen des Templates verwendet werden. Diese Datenbindungen erhalten ihren Kontext direkt von der Komponente und bilden somit das Bindeglied zwischen den Direktiven und dem restlichen Anwendungscode.²⁰

Komponenten definieren nicht nur die Struktur der Benutzeroberfläche, sondern auch deren Verhalten. Sie ermöglichen es, komplexe Anwendungen in übersichtliche, wiederverwendbare und wartbare Einheiten zu unterteilen. Gerade in größeren Angular-Projekten sind Komponenten ein wichtiges Werkzeug, um die Anwendung in modulare und gut organisierte Teilbereiche zu gliedern.²¹ Durch diese klare Trennung von Darstellung und Logik fördern Komponenten die Lesbarkeit, Testbarkeit und Skalierbarkeit von Angular-Anwendungen.

2.8 Data binding

Bei der Entwicklung einer Anwendung, die dem Benutzer Daten anzeigen und gleichzeitig Änderungen erfassen soll, bietet sich ein strukturierter Ansatz an. Zunächst werden die Daten geladen und für jedes einzelne Element wird ein entsprechender Abschnitt in der Benutzeroberfläche erzeugt. Dabei kommen wiederholende Komponenten wie Textfelder, Dropdown-Menüs oder andere Eingabeelemente zum Einsatz, die die jeweiligen Daten darstellen.²²

Sobald der Benutzer Anpassungen vorgenommen hat, wird die Oberfläche erneut durchlaufen. Dabei wird geprüft, ob sich die Inhalte der Eingabeelemente verändert haben. Falls Änderungen festgestellt werden, werden die neuen Werte zurück in die ursprünglichen Datenobjekte übertragen, um sie anschließend speichern zu können.²³

¹⁸ Adam Freeman, Pro Angular 16, Kapitel 16.

¹⁹ Adam Freeman, Pro Angular 16, Kapitel 16.

²⁰ Adam Freeman, Pro Angular 16, Kapitel 16.

²¹ Adam Freeman, Pro Angular 16, Kapitel 16.

²² Peter Himschoot, Microsoft Blazor, Third Edition, Kapitel 2, Seite 51.

²³ Peter Himschoot, Microsoft Blazor, Third Edition, Kapitel 2, Seite 51.

Dieser manuelle Ansatz ist fehleranfällig und mit viel Aufwand verbunden, insbesondere wenn man dafür eine Bibliothek wie jQuery verwendet. jQuery ist ein weit verbreitetes JavaScript-Framework, das die Manipulation des Document Object Model (DOM) im Browser ermöglicht.

Moderne Frameworks wie Angular erfreuen sich großer Beliebtheit, weil sie diesen Prozess durch Datenbindung erheblich vereinfachen. Mithilfe der Datenbindung übernimmt das Framework den Großteil der Arbeit. Die Benutzeroberfläche wird automatisch aus den Daten generiert und Änderungen durch den Benutzer werden direkt in die zugrunde liegenden Datenobjekte zurückgeschrieben, ganz ohne manuelles Auslesen und Aktualisieren.²⁴

2.8.1 Grundlage der Datenbindung (Data Binding)

In modernen Webframeworks wie Angular spielt die Datenbindung eine zentrale Rolle bei der dynamischen Darstellung und Aktualisierung von Inhalten in Webanwendungen.

Dabei handelt es sich um Ausdrücke, die direkt in Templates eingebettet werden. Diese werden zur Laufzeit ausgewertet und erzeugen dynamische Inhalte im HTML-Dokument. So können sich angezeigte Daten automatisch aktualisieren, sobald sich die zugrunde liegenden Werte im Anwendungscode ändern, ganz ohne manuelle DOM-Manipulation. Datenbindungen schaffen eine direkte Verbindung zwischen den HTML-Elementen und den Daten sowie der Logik der Anwendung. Sie werden entweder als Attribute in HTML-Elementen oder als spezielle Zeichenfolgen innerhalb von Strings verwendet. Die enthaltenen Ausdrücke basieren meist auf einfachen JavaScript-Ausdrücken, die zur Generierung von Inhalten dienen.²⁵

Ein häufiger Fehler besteht darin, zu viel Logik in eine Bindung einzubauen. Solche komplexen Ausdrücke lassen sich nur schwer testen und wiederverwenden. Daher sollten Datenbindungsausdrücke möglichst einfach gehalten werden. Der Einsatz von Signals kann zusätzlich helfen, Änderungen effizient zu erkennen und die Struktur übersichtlich zu halten.²⁶

Insgesamt ist die Datenbindung ein unverzichtbares Werkzeug in der Angular-Entwicklung, das die Erstellung interaktiver und reaktiver Benutzeroberflächen erheblich vereinfacht.²⁷

2.8.2 One-way Data Bindings

Einseitige Datenbindung (One-way Data Binding) ist eine grundlegende Funktion in Angular-Templates und wird verwendet, um Inhalte für den Benutzer darzustellen. Der Begriff „einseitig“ bezieht sich darauf, dass der Datenfluss nur in eine Richtung verläuft, und zwar von der Komponente zur Benutzeroberfläche. Das bedeutet, dass die Daten aus dem Anwendungscode in das Template übertragen werden, wo sie angezeigt werden, jedoch nicht umgekehrt. Änderungen in der Benutzeroberfläche wirken sich bei dieser Bindungsart nicht automatisch auf die zugrunde liegenden Daten aus. Diese Art der Datenbindung eignet sich besonders gut für die Anzeige von statischen oder berechneten Werten, bei denen direkte Rückmeldung vom Benutzer erforderlich ist. Sie sorgt für eine klare Trennung zwischen Datenquelle und ist leicht zu implementieren und zu testen.²⁸

2.8.3 Event-Binding

Event Bindings ermöglichen die Ausführung von Ausdrücken, sobald ein bestimmtes Ereignis ausgelöst wird, etwas durch das Drücken einer Taste, das Bewegen der Maus oder das Absenden eines Formulars.

²⁴ Peter Himschoot, Microsoft Blazor, Third Edition, Kapitel 2, Seite 51.

²⁵ Adam Freeman, Pro Angular 16, Kapitel 11.

²⁶ Adam Freeman, Pro Angular 16, Kapitel 11.

²⁷ Adam Freeman, Pro Angular 16, Kapitel 11.

²⁸ Adam Freeman, Pro Angular 16, Kapitel 11.2

Diese Bindungen bilden die Grundlage für viele interaktive Funktionen in Angular-Anwendungen. Darauf aufbauend bietet Angular erweiterte Formularfunktionen, die es ermöglichen, automatisch validierte Formulare zu erstellen. So wird sichergestellt, dass der Benutzer sinnvolle und korrekte Daten eingibt, bevor diese verarbeitet werden.²⁹

Diese Funktionen erlauben es dem Benutzer, den Zustand der Anwendung aktiv zu verändern, etwa durch das Ändern oder Hinzufügen von Werten im Datenmodell.

In meinem Projekt habe ich Event Binding genutzt, um Benutzereingaben zu erfassen und die Berechnung dynamisch auszulösen. Sobald ein Wert eingegeben oder geändert wird, reagiert die Anwendung sofort, aktualisiert das Modell und zeigt das Ergebnis direkt in der Benutzeroberfläche an.

2.8.4 Two-way Data Bindings

Datenbindungen können miteinander kombiniert werden, um einen bidirektionalen Datenfluss für ein einzelnes Element zu ermöglichen. Dabei reagiert das HTML-Dokument automatisch auf Änderungen im Anwendungsmodell und gleichzeitig kann die Anwendung auf Benutzereingaben reagieren, die durch Ereignisse im HTML-Element ausgelöst werden.³⁰

Diese Form der Datenbindung erlaubt eine synchronisierte Kommunikation zwischen Benutzeroberfläche und Anwendungscode. Ein typisches Beispiel ist die Eingabe in einem Formularfeld: Wenn der Benutzer einen Wert eingibt, wird dieser direkt im Datenmodell gespeichert. Ändert sich der Wert im Modell, wird er automatisch im Eingabefeld aktualisiert.

In meinem Projekt, dem Viskositätsindex-Rechner, habe ich genau dieses Prinzip mithilfe eines Formulars namens viForm umgesetzt. Dabei handelt es sich um ein reaktives Formular, das mit Angulars FormGroup und FormControl aufgebaut ist. Die Eingabefelder für z.B. Viskosität bei 40°C, Viskosität bei 100°C oder andere relevante Parameter sind über Two-Way Data Binding mit dem Datenmodell verbunden. Sobald der Benutzer einen Wert eingibt oder ändert, wird dieser automatisch im Formularmodell (viForm.value) aktualisiert. Gleichzeitig kann ich über das Modell auf die aktuellen Werte zugreifen, um z.B. den Viskositätsindex in Echtzeit zu berechnen und das Ergebnis direkt im Template anzuzeigen. Dadurch entsteht eine nahtlose Interaktion zwischen Benutzer und Anwendung, ohne zusätzliche manuelle DOM-Abfragen oder Event-Handler.

2.9 Formulare

Webanwendungen nutzen Formulare, um Eingabedaten von Benutzern zu erfassen. In meinem Projekt, dem Viskositätsindex-Rechner steht genau diese Funktion im Mittelpunkt. Benutzer geben physikalische Werte wie die Viskosität bei 40°C und 100°C über ein Eingabeformular ein, um den Viskositätsindex zu berechnen. Die erfassten Daten werden im Formularmodell (viForm) gespeichert, validiert und können bei Bedarf auch persistiert werden, bspw. zur Zwischenspeicherung oder zur Weiterverarbeitung über eine Schnittstelle im Backend.³¹ Diese Funktionen ermöglichen es dem Benutzer, aktiv mit der Anwendung zu interagieren, Daten zu verändern und den Zustand der Anwendung zu beeinflussen.

Ein Formular in einer Webanwendung besitzt typischerweise bestimmte Merkmale, die die Benutzererfahrung deutlich verbessern:³²

²⁹ Adam Freeman, Pro Angular 16, Kapitel 13.1.

³⁰ Adam Freeman, Pro Angular 16, Kapitel 13.3.

³¹ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

³² Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

- Definition verschiedener Eingabefelder: Ein Formular kann unterschiedliche Feldtypen enthalten z.B. Zahlenfelder, Dropdowns oder Textfelder, je nach Art der benötigten Daten. Im Viskositätsindex-Rechner werden bspw. numerische Felder für die Eingabe der Viskosität bei 40°C und 100°C verwendet.
- Einrichtung von Validierungen und Anzeige von Fehlermeldungen: Um sicherzustellen, dass die eingegebenen Daten korrekt und sinnvoll sind, werden Validierungsregeln definiert. Bei ungültigen Eingaben erhält der Benutzer sofort eine visuelle Rückmeldung, z.B. in Form von Fehlermeldungen oder farblichen Markierungen.
- Unterstützung verschiedener Strategien zur Fehlerbehandlung: Wenn sich das Formular in einem Fehlerzustand bspw. durch fehlende oder ungültige Eingaben befindet, kann die Anwendung unterschiedliche Maßnahmen ergreifen.

Diese Merkmale sorgen dafür, dass der Benutzer gezielt durch die Eingabe geführt wird, Fehler frühzeitig erkennt und die Anwendung verlässlich und intuitiv kann, ein entscheidender Faktor für die Qualität und Benutzerfreundlichkeit des Viskositätsindex-Rechners.

Das Angular-Framework bietet zwei unterschiedliche Ansätze zur Umsetzung von Formularen, nämlich Template-driven Forms und Reactive Forms. Keiner der beiden Ansätze ist grundsätzlich besser und die Wahl hängt vom jeweiligen Anwendungsszenario ab.³³

2.9.1 Template-driven Forms

Template-driven Formulare in Angular erweitern die bisherige Arbeitsweise mit Komponenten und erinnern in ihrer Struktur und Syntax stark an die Formularlogik aus AngularJS. Entwickler:innen mit entsprechender Vorerfahrung finden sich daher schnell zurecht. Der Ansatz basiert auf der Vorlage (Template) und nutzt die Datenbindung, um Informationen zwischen dem Formular und der Komponente auszutauschen. Die Validierungs- und Steuerungslogik wird direkt im HTML definiert, wodurch sich dieser Ansatz besonders für einfache Formulare eignet.³⁴

Template-basierte Formulare sind einfach einzurichten und lassen sich schnell in eine Angular-Anwendung integrieren. Die gesamte Logik, einschließlich der Erstellung von Eingabefeldern und Validierungsregeln, wird direkt im Template definiert. Dadurch sind sie besonders für kleinere, einfache Formulare geeignet. Allerdings gestaltet sich das Testen solcher Formulare schwieriger, da sie stark vom Change Detection Mechanismus des Frameworks abhängig sind.³⁵

2.9.2 Reactive Forms

Bevor man sich mit der Erstellung reaktiver Formulare in Angular beschäftigt, lohnt sich ein Blick auf das zugrunde liegende Konzept der reaktiven Programmierung. Dabei handelt es sich um einen Ansatz, bei dem Anwendungen gezielt auf asynchrone Datenströme reagieren. Anstatt Daten nur einmalig zu verarbeiten, wird kontinuierlich auf Veränderungen geantwortet, etwa bei Benutzereingaben oder externen Ereignissen.

Reaktive Programmierung stellt dafür leistungsstarke Werkzeuge bereit, mit denen sich Datenströme verknüpfen, filtern und kombinieren lassen. Diese Dynamik macht sie besonders geeignet für komplexe Webanwendungen, wie etwa die Formularverarbeitung in Angular mit Reaktive Formularen.³⁶

Im Gegensatz zu template-gesteuerten Formularen wird bei reaktiven Formularen der gesamte Baum der Formularsteuerelemente im Komponentencode definiert. Die einzelnen FormControl-Elemente

³³ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

³⁴ Shyam Seshadri, Angular: Auf und davon, Kapitel 6.

³⁵ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

³⁶ Shyam Seshadri, Angular: Auf und davon, Kapitel 6.

werden anschließend über Datenbindung mit den nativen HTML-Formularfeldern im Template verknüpft. Da die Komponente direkten Zugriff sowohl auf die Formularkontrollen als auch auf das zugrunde liegende Datenmodell hat, können Änderungen in beide Richtungen übertragen werden. Das bedeutet: Änderungen am Modell wirken sich auf die Anzeige im Formular aus und umgekehrt. Diese bidirektionale Reaktivität ermöglicht eine präzise Steuerung und dynamische Validierung, was reaktive Formulare besonders leistungsfähig macht.³⁷

Reaktive Formulare sind besonders dann sinnvoll, wenn es um skalierbare und testbare Anwendungen geht. Die Formularlogik wird vollständig in der Komponentenklasse definiert. Eingabefelder, Validierungen und Datenverarbeitung erfolgen über ein Formularmodell, das unveränderlich bleibt und sich gut in reaktive Programmieransätze integrieren lässt. Dieser Ansatz eignet sich besonders für Anwendungen mit komplexen Formularstrukturen oder vielen dynamischen Eingabefeldern.³⁸

Reaktive Formulare bieten eine reaktive Schnittstelle zur Arbeit mit Webformularen. Sie sind speziell für Anwendungen konzipiert, die auf Reaktivität und Datenströme setzen. Die Eingabefelder und ihre Werte werden über Observable Streams verwaltet, was eine flexible und dynamische Steuerung der Formulardaten ermöglicht.³⁹

Ein zentraler Vorteil reaktiver Formulare ist die Unveränderlichkeit des Formularzustands. Änderungen am Formular erfolgen explizit und kontrolliert, wodurch sich der Zustand der Daten konsistent und nachvollziehbar verändern lässt. Dies erleichtert insbesondere das Testen, da man sich darauf verlassen kann, dass jede Änderung gezielt vorgenommen wird und keine ungewollten Seiteneffekte entstehen.⁴⁰

In meinem Projekt habe ich diesen Ansatz gewählt, um die Benutzereingaben zuverlässig zu erfassen, Validierungen sauber umzusetzen und die Berechnung des Viskositätsindex auf Basis der aktuellen Formulardaten durchzuführen. Die reaktive Struktur erlaubt es mir, jederzeit auf Änderungen zu reagieren und die Benutzeroberfläche entsprechend zu aktualisieren, was ein entscheidender Vorteil für eine präzise und benutzerfreundliche Anwendung.

Reactive Forms verfolgen einen programmgesteuerten Ansatz zur Erstellung von Formularelementen und zur Definition von Validierungsregeln. Dabei wird die gesamte Formularlogik in der Komponentenklasse festgelegt, nicht in Template. Dies bietet eine hohe Flexibilität und erleichtert die Wartung und das Testen der Anwendung.⁴¹

Angular stellt dafür eine Reihe von Schlüsselklassen zur Verfügung, die den Aufbau und die Verwaltung reaktiver Formulare ermöglichen:⁴²

- **FormControl**: repräsentiert ein einzelnes Eingabefeld im Formular. Es speichert den aktuellen Wert und den Status und ermöglicht die Definition von Validierungsregeln.
- **FormGroup**: Eine Sammlung von FormControl-Instanzen, die gemeinsam verwaltet werden. Sie bildet die Struktur des gesamten Formulars ab und erlaubt die zentrale Steuerung von Validierungen und Zuständen.
- **FormBuilder**: Eine Hilfsklasse, die die Erstellung von FormGroup und FormControl vereinfacht. Sie reduziert den Codeaufwand und verbessert die Lesbarkeit.

³⁷ Shyam Seshadri, Angular: Auf und davon, Kapitel 7.

³⁸ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

³⁹ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

⁴⁰ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

⁴¹ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

⁴² Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

- **Validators:** Eine Sammlung vordefinierter Validierungsfunktionen, die auf FormControl-Elemente angewendet werden können, um Eingaben zu überprüfen.

2.9.3 Gegenüberstellung von Template-driven und Reactive Forms

Angular bietet zwei grundlegende Strategien zur Erstellung von Formularen, nämlich template-driven Formulare und reaktive Formulare. Welche Ansatz geeigneter ist, hängt vom konkreten Anwendungsfall ab.

Bei template-driven Formularen wird die Formularstruktur direkt im HTML-Template definiert. Angular übernimmt dabei die interne Verwaltung der Steuerelemente über Direktiven wie ngModel. Diese Methode ist besonders deklarativ und eignet sich gut für einfache Formulare mit geringem Logikanteil. Die Synchronisierung zwischen UI und Datenmodell erfolgt automatisch über die Direktiven.⁴³

Bei reaktiven Formularen wird der gesamte Formularbaum explizit in der TypeScript-Klasse der Komponente aufgebaut. Der Entwickler hat volle Kontrolle über Struktur, Validierung und Datenfluss. Die Synchronisierung zwischen UI und Datenmodell erfolgt manuell und gezielt. Da die FormControls direkt verfügbar sind, lassen sich Änderungen sofort umsetzen, ohne auf den Lebenszyklus von Angular warten zu müssen. Dieser Ansatz eignet sich besonders für komplexe Anwendungen und passt gut zu einem reaktiven Programmierstil.

2.10 Validierung von Eingaben in Formularen

Die Validierung von Benutzereingaben ist ein zentraler Bestandteil moderner Webanwendungen, insbesondere wenn es um die Qualität und Sicherheit der Daten geht. Angular bietet verschiedene Möglichkeiten zur Implementierung von Validierungslogiken, die sowohl technische als auch benutzerfreundliche Anforderungen erfüllen.

Grundsätzlich unterscheidet man in Angular zwischen folgenden Validierungsansätzen:

2.10.1 Globale CSS-Validierung:⁴⁴

Angular vergibt automatisch bestimmte CSS-Klassen an Formulare und deren Eingabefelder, abhängig vom aktuellen Status der Validierung. Diese Klassen dienen der visuellen Rückmeldung und unterstützen die Benutzerführung:

- **ng-valid/ng-invalid:** zeigt an, ob der aktuelle Wert eines Formularfeldes gültig oder ungültig ist.
- **ng-touched/ng-untouched:** zeigt an, ob der Benutzer bereits mit dem Feld interagiert hat (ng-touched) oder noch nicht (ng-untouched).
- **ng-dirty/ng-pristine:** zeigt an, ob der Wert eines Feldes verändert wurde (ng-dirty) oder sich noch im ursprünglichen Zustand befindet (ng-pristine).

Die globale CSS-Validierung wird in dieser Arbeit nicht tiefgehend behandelt, da ich mich auf die Umsetzung mit Reactive Form Validation und Custom Validation konzentriert habe.

2.10.2 Validierung in Template-driven Forms

Angular verfügt über eigene integrierte Validatoren, die dem Verhalten von HTML-Formular-Validatoren entsprechen. Diese werden innerhalb einer Angular-Anwendung verwendet, um eine vergleichbare Validierungslogik umzusetzen. Sobald einem Formularelement Validatoren zugewiesen sind, stellt Angular sicher, dass diese bei jeder Änderung des Steuerelements automatisch ausgeführt

⁴³ Shyam Seshadri, Angular: Auf und davon, Kapitel 7.

⁴⁴ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

werden. Die Validierung erfolgt dabei sowohl auf der Ebene des einzelnen Formularfeldes als auch auf der Ebene des gesamten Formulars.⁴⁵

2.10.3 Validierung in Reactive Forms

In reaktiven Formularen liegt die zentrale Steuerung in der Komponentenklasse. Die Validierungslogik wird nicht über HTML-Attribute im Template definiert, sondern direkt im Formularmodell innerhalb der Komponente. Angular führt diese Funktionen automatisch aus, sobald sich der Wert eines Formularfeldes ändert. Dadurch wird die Validierung kontinuierlich und zuverlässig auf dem aktuellen Stand gehalten, sowohl auf der Ebene einzelner Felder also auch für das gesamte Formular.⁴⁶

Validator-Funktionen können entweder synchron oder asynchron sein.

- Synchroner Validatoren sind Funktionen, die direkt beim Aufruf eine Instanz eines Formularfeldes (FormControl) prüfen und dabei entweder ein Objekt mit Validierungsfehlern oder null zurückgeben. Sie werden beim Erstellen einer FormControl als zweites Argument übergeben und ermöglichen eine sofortige Überprüfung der Eingabewerte.⁴⁷
- Asynchrone Validatoren sind Funktionen, die eine Kontrollinstanz entgegennehmen und ein Promise oder ein Observable zurückgeben. Dieses gibt zu einem späteren Zeitpunkt entweder ein Objekt mit Validierungsfehlern oder null aus. Solche Validatoren werden beim Erstellen einer FormControl als drittes Argument übergeben und eignen sich besonders für Validierungen, die externe Datenquellen einbeziehen.⁴⁸

Aus Performance-Gründen führt Angular asynchrone Validatoren nur dann aus, wenn alle synchronen Validatoren erfolgreich sind, also keine Fehler zurückgeben. Jeder Validator muss vollständig abgeschlossen sein, bevor Angular die Validierungsfehler im Formularfeld setzt.⁴⁹ Dieses Verhalten sorgt dafür, dass unnötige asynchrone Prüfungen vermieden werden und die Validierung effizient bleibt.

2.10.4 Benutzerdefinierte Validierung

Die eingebauten Validatoren in Angular decken nicht alle Anwendungsfälle ab, die in einer Anwendung auftreten können. Dennoch ist das Schreiben eines benutzerdefinierten Validators und dessen Verwendung in einem Angular-Formular einfach umzusetzen.⁵⁰

Ein Formular-Validator ist eine Funktion, die zur Validierung von Benutzereingaben in einem Formular verwendet wird. Sie erhält eine Instanz eines Formularfeldes (FormControl) als Parameter und gibt entweder ein Objekt mit Validierungsfehlern oder null zurück. Wird ein Fehler festgestellt, enthält das zurückgegebene Objekt Informationen über die Art des Fehlers. Ist die Eingabe gültig, liefert die Funktion null. Diese Struktur ermöglicht eine flexible und erweiterbare Validierungslogik, die sowohl einfache als auch komplexe Prüfungen abbilden kann.⁵¹

⁴⁵ Shyam Seshadri, Angular: Auf und davon, Kapitel 6.

⁴⁶ <https://angular.dev/guide/forms/form-validation#validating-input-in-reactive-forms>, Zugriff am: 13.08.2025.

⁴⁷ <https://angular.dev/guide/forms/form-validation#validating-input-in-reactive-forms>, Zugriff am: 13.08.2025.

⁴⁸ <https://angular.dev/guide/forms/form-validation#validating-input-in-reactive-forms>, Zugriff am: 13.08.2025.

⁴⁹ <https://angular.dev/guide/forms/form-validation#validating-input-in-reactive-forms>, Zugriff am: 13.08.2025.

⁵⁰ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

⁵¹ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

Der Bezeichner im Validierungsfehler-Objekt dient dazu, die Art des Fehlers eindeutig zu benennen. Dieser Name kann später verwendet werden, um mithilfe der Methode `hasError` zu prüfen, ob ein bestimmter Validierungsfehler vorliegt. Der zugehörige Wert kann beliebig gewählt werden und enthält zusätzliche Informationen, die beispielsweise in einer Fehlermeldung angezeigt oder für die weitere Verarbeitung genutzt werden können.⁵²

Bei der Validierung von Eingabedaten im Viskositätsindex-Rechner spielt nicht nur die technische Umsetzung eine Rolle, sondern auch die Einhaltung fachlicher Standards. Die Berechnung des Viskositätsindex basiert auf der ASTM D2270-Norm, die international als Standardverfahren zur Bestimmung des Viskositätsindex von Schmierstoffen anerkannt ist.

Diese Norm definiert unter anderem:

- Welche Wertebereich für die Viskosität bei 40°C und 100°C zulässig und sinnvoll sind.
- Welche mathematische Formel zur Berechnung verwendet wird.
- Welche Genauigkeit bei der Eingabe und Verarbeitung der Daten erforderlich ist.

Im Rahmen der Validierung wurden diese Vorgaben berücksichtigt, um sicherzustellen, dass nur normgerechte und plausible Werte eingegeben werden können. So wird z.B. geprüft, ob die Viskosität bei 100°C nicht höher ist als bei 40°C, ein typischer Fehler, der gegen die physikalischen Grundlagen und die ASTM-Vorgaben verstoßen würde.⁵³

Durch die Einbindung der ASTM-Norm in die Validierungslogik wird gewährleistet, dass die Berechnung nicht nur technisch korrekt, sondern auch fachlich fundiert erfolgt.

2.11 Einführung in Angular Material

Bei der Entwicklung einer Webanwendung muss entschieden werden, wie die Benutzeroberfläche (UI) gestaltet wird. Eine gut umgesetzte UI sollte kontrastreiche Farben verwenden, ein einheitliches Erscheinungsbild bieten, responsiv sein und auf verschiedenen Geräten sowie in unterschiedlichen Browsern zuverlässig funktionieren. Kurz gesagt: Es gibt viele Aspekte, die bei der Gestaltung von UI und UX (User Experience) berücksichtigt werden müssen. Da die Umsetzung dieser Anforderungen oft komplex ist, greifen viele Entwickler auf UI-Frameworks zurück, die einen Großteil der gestalterischen und technischen Arbeit übernehmen.⁵⁴

Zu den verbreitetsten Frameworks zählen Bootstrap und Tailwind CSS. In den letzten Jahren hat jedoch auch Angular Material zunehmend an Bedeutung gewonnen. Dieses Framework basiert auf den Material Design-Richtlinien von Google und stellt eine umfangreiche Komponentenbibliothek für Angular bereit.⁵⁵

In diesem Kapitel wird erläutert, was Material Design ist und wie Angular Material diese Prinzipien nutzt, um die Entwicklung moderner und benutzerfreundlicher Oberflächen zu erleichtern.

2.11.1 Material Design

Material Design ist eine von Google entwickelte Designsprache mit dem Ziel, ein einheitliches System zu schaffen, das eine konsistente Benutzererfahrung über verschiedene Plattformen und Gerätegrößen hinweg ermöglicht. Die Grundprinzipien orientieren sich stark an mobilen Anwendungen, jedoch werden alle Eingabemethoden, darunter Touch, Spracheingabe, Maus und Tastatur als gleichwertig

⁵² Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 10.

⁵³ ASTM D2270-24, Standard Practice for Calculating Viscosity Index from Kinematic Viscosity at 40°C and 100°C.

⁵⁴ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 12.

⁵⁵ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 12.

betrachtet und unterstützt.⁵⁶ Material Design legt besondere Wert auf Klarheit, visuelle Hierarchie, intuitive Interaktionen und eine moderne, reduzierte Ästhetik.

2.11.2 Angular Material

Die Angular Material Library wurde entwickelt, um die Prinzipien von Material Design direkt im Angular-Framework umzusetzen. Sie basiert auf folgenden zentralen Konzepten:⁵⁷

- Schneller Einstieg: Ziel ist es, Entwickler*innen einen einfachen und schnellen Start zu ermöglichen. Der Aufwand für die Einrichtung soll minimal sein, sodass man direkt mit der Entwicklung beginnen kann.
- Leistungsstark und konsistent: Die Performance steht im Fokus. Angular Material ist darauf ausgelegt, zuverlässig und schnell auf allen gängigen Browsern zu funktionieren.
- Vielseitigkeit: Die Bibliothek bietet eine breite Auswahl an Themes, die sich leicht anpassen lassen. Zudem wird Internationalisierung und Lokalisierung umfassend unterstützt.
- Optimierte für Angular: Da Angular Material direkt vom Angular-Team entwickelt wurde, ist die nahtlose Integration in Angular-Anwendungen gewährleistet.

Die Angular Material Library ist in zwei Hauptbereiche unterteilt: Komponenten zur Gestaltung der Benutzeroberfläche und Themes zur Anpassung des Designs.⁵⁸

Das Herzstück von Angular Material bildet das Angular CDK (Component Dev Kit). Es handelt sich um eine Sammlung von Werkzeugen zur Umsetzung von Interaktionsmustern, die unabhängig vom visuellen Design sind. Die Komponenten von Angular Material basieren funktional auf dem CDK, das so abstrakt gestaltet ist, dass es sich auch für die Entwicklung eigener UI-Komponenten eignet.⁵⁹

3. Grundlagen des Viskositätsindex

Der Viskositätsindex (VI) gibt an, wie stark sich die Viskosität eines Schmieröls bei Temperaturänderungen verändert. Je höher der Viskositätsindex, desto stabiler bleibt die Viskosität bei unterschiedlichen Temperaturen. Ein Öl mit einem hohen VI eignet sich besonders für Anwendungen, bei denen große Temperaturschwankungen auftreten, da es sowohl bei Kälte als auch bei Hitze eine zuverlässige Schmierung bietet.

Die Berechnung des VI erfolgt nach einem standardisierten Verfahren gemäß der ASTM D2270-Norm. Dabei wird die kinematische Viskosität bei 40°C und 100°C gemessen und mit Referenzölen verglichen.⁶⁰ Die genaue Berechnungslogik sowie die zugrunde liegende Formel werden im nächsten Kapitel näher erläutert.

4. Praxisanteil: Entwicklung des Features

Im folgenden Kapitel wird die praktische Umsetzung des Viskositätsindex-Rechners vorgestellt. Ziel ist es, eine benutzerfreundliche Webanwendung zu entwickeln, die auf Basis der kinematischen Viskositäten bei 40°C und 100°C den Viskositätsindex gemäß der ASTM D2270-Norm berechnet. Dabei werden sowohl die technische Umsetzung mit Angular als auch die zugrunde liegende

⁵⁶ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 12.

⁵⁷ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 12.

⁵⁸ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 12.

⁵⁹ Aristeidis Bampakos, Learning Angular, 5th Edition, Kapitel 12.

⁶⁰ ASTM D2270-24, Standard Practice for Calculating Viscosity Index from Kinematic Viscosity at 40°C and 100°C.

Berechnungslogik erläutert. Der Fokus liegt auf der Verbindung von theoretischem Wissen mit praktischer Anwendung.

4.1 Validierung der Benutzereingaben

Ein wichtiger Bestandteil der Umsetzung ist die Validierung der Benutzereingaben, um physikalisch sinnvolle Werte sicherzustellen. Dazu wurde eine benutzerdefinierte Validierungsfunktion namens „relativeValidatorFactory“ implementiert. Diese prüft, ob die Viskosität bei 40°C größer als die bei 100°C ist.

```
function relativeValidatorFactory(  
  relativeControl: FormControl,  
  comparison: '>' | '<'  
) : ValidatorFn {  
  return (control: AbstractControl): ValidationErrors => {  
    if (comparison == '<' && !(control.value < relativeControl.value)) {  
      return { relativeValidatorError: true };  
    }  
    if (comparison == '>' && !(control.value > relativeControl.value)) {  
      return { relativeValidatorError: true };  
    }  
    return {};  
  };  
}
```

Diese Funktion wird dynamisch auf beide Eingabefelder angewendet und sorgt dafür, dass fehlerhafte oder unlogische Eingaben direkt im Formular erkannt und verhindert werden.

4.2 Formularstruktur und Eingabefelder

Für die Eingabe der kinematischen Viskositäten wurden ein reaktives Formular mit zwei Feldern erstellt.

```
viValue: number | null = null;  
formSubmitted = false;  
  
v40 = new FormControl<number>(null, Validators.required);  
v100 = new FormControl<number>(null, [  
  Validators.required,  
  Validators.min(3),  
]);  
viForm: FormGroup = new FormGroup({ v40: this.v40, v100: this.v100 });
```

- V40: Eingabefeld für die Viskosität bei 40°C, Pflichtfeld.
- V100: Eingabefeld für die Viskosität bei 100°C, Pflichtfeld mit zusätzlicher Mindestanforderung (min(3)), da sehr niedrige Werte laut ASTM D2270-Norm nicht zulässig sind.
- viForm: Das übergeordnete Formularobjekt, das beide Felder zusammenfasst und deren Validierung steuert.

Die Variable viValue speichert das Ergebnis der Berechnung, während formSubmitted den Status des Formulars verfolgt. Diese Struktur bildet die Grundlage für die Interaktion mit dem Benutzer und die spätere Berechnung des Viskositätsindex.

4.3 Berechnung des Viskositätsindex

Die zentrale Berechnung erfolgt in der Methode `calculateVI()`, die beim Absenden des Formulars aufgerufen wird. Sie prüft zunächst, ob das Formular gültig ist und ruft anschließend die Funktion `calculateVIFormula(v40, v100)` auf, um den Viskositätsindex zu berechnen.

```
calculateVI(): void {
  this.formSubmitted = true;

  if (!this.viForm || this.viForm.invalid) {
    this.viValue = null;
    return;
  }

  // v40: kinematic viscosity of the lubricant at 40°C
  const v40 = this.viForm.value.v40;
  // v100: kinematic viscosity of the lubricant at 100°C
  const v100 = this.viForm.value.v100;
  this.viValue = this.calculateVIFormula(v40, v100);
}
```

Die eigentliche Berechnungslogik basiert auf der ASTM D2270-Norm und unterscheidet zwei Fälle:

- Für hohe Viskositätswerte ($v_{100} > 70 \text{ mm}^2/\text{s}$) werden die Referenzwerte L und H direkt über eine mathematische Formel berechnet.
- Für niedrigere Werte werden L und H aus einer vorbereiteten Tabelle (`viscosityTable`) entnommen.

```
let L: number, H: number;
if (v100 <= 2) {
  return NaN;
}

if (v100 > 70) {
  L = 0.8353 * Math.pow(v100, 2) + 14.67 * v100 - 216;
  H = 0.1684 * Math.pow(v100, 2) + 11.85 * v100 - 97;
} else {
  const row = viscosityTable.find((r) => r.v100 === Number(v100));
  if (!row) {
    return NaN;
  }
  L = row.L;
  H = row.H;
}
```

Anschließend wird je nach Verhältnis von v_{40} zu H entweder eine lineare oder logarithmische Formel verwendet.

```
if (v40 > H) {
  const VI = 100 * ((L - v40) / (L - H));
  return Math.round(VI);
} else {
  const N = (Math.log(H) - Math.log(v40)) / Math.log(v100);
  const VI = 100 + (Math.pow(10, N) - 1) / 0.00715;
  return Math.round(VI);
}
```

Diese Berechnung liefert den Viskositätsindex, der anschließend im Interface angezeigt wird. Durch die Kombination aus mathematischer Logik und Benutzerinteraktion entsteht eine praxisnahe Anwendung, die die Theorie direkt erlebbar macht.

4.4 Integration in die Benutzeroberfläche

Der Viskositätsindex-Rechner wurde als eigenständiges Modul in die bestehende Grease App integriert. In der Benutzeroberfläche erscheint er als App-Ad-Card mit dem Titel „Viscosity Index Calculator“. Diese Komponente ist klar strukturiert und enthält:

- Eine kurze Beschreibung der Funktion.
- Einen Button mit der Aufschrift „VI-Calculator“, der zur Eingabemaske führt.

Diese Integration macht den Rechner leicht zugänglich und unterstützt die Nutzer:innen dabei, technische Entscheidungen direkt in der App zu treffen, ohne zusätzliche Hilfsmittel.

The screenshot displays the Grease App interface. At the top, there is a green gear icon with a drop and the text "Grease App". Below this, a green badge says "New" followed by "Over 100 greases available for interchangeability". A search bar contains the placeholder text "Enter a bearing designation, e.g. 6226 or a grease name" and a magnifying glass icon. Below the search bar, a green link says "Advanced bearing selection".

The main content area features a card titled "New VISCOSITY INDEX CALCULATOR". The card text reads: "With our integrated viscosity index calculator, you can quickly and easily calculate the viscosity index of a lubricant. The tool provides precise results and helps you to reliably assess the temperature stability of lubricants." To the right of the text is a button labeled "VI-Calculator".

A modal window is open over the card, showing the input fields for the calculator. It has two input fields: "Kinematic Viscosity at 40°C (v40)*" with the value "80" and unit "mm²/s", and "Kinematic Viscosity at 100°C (v100)*" with the value "50" and unit "mm²/s". Below these fields, it says "Viscosity Index (VI):" followed by a large box displaying the result "549".

At the bottom of the app, there is a feedback section with the text "Your feedback helps us to continuously improve our product." and a "Feedback" button.

Die Eingabemaske für den Viskositätsindex-Rechner wurde als Dialogfenster innerhalb der Homepage der Grease App umgesetzt und nicht als separate Seite. Der Hintergrund dieser Entscheidung liegt in der kompakten Struktur der Eingabefelder und dem vorhandenen unbesetzten Weißraum auf der Benutzeroberfläche. Um diesen Platz effizient zu nutzen und die Eingabe visuell hervorzuheben, wurde die Dialogdarstellung gewählt.

Durch diese Lösung bleibt die Hauptseite der App sichtbar, während die Eingabe und Berechnung des VI fokussiert um Dialog erfolgt. Nutzer:innen geben dort die kinematische Viskosität bei 40°C und 100°C ein. Sobald beide Werte gültig sind, wird der berechnete Viskositätsindex (VI) automatisch im Dialog angezeigt.

5. Reflexion und Ausblick

Die Umsetzung des Viskositätsindex-Rechners als Webanwendung innerhalb der Grease App war ein zentraler Bestandteil dieser Projektarbeit. Ziel ist es, eine funktionale und benutzerfreundliche Lösung zu entwickeln, die die Berechnung des VI gemäß ASTM D2270 ermöglicht und sich nahtlos in die bestehende App-Struktur integriert.

5.1 Bewertung der Umsetzung

Die technische Umsetzung mit Angular und reaktiven Formularen hat sich als effektiv erwiesen. Besonders positiv hervorzuheben ist die Entscheidung, die Eingabemaske als Dialog innerhalb der Homepage darzustellen, statt eine separate Seite zu verwenden. Diese Lösung nutzt den vorhandenen Weißraum sinnvoll aus und ermöglicht eine fokussierte Interaktion, ohne die Hauptnavigation zu verlassen. Die automatische Berechnung des VI bei gültigen Eingaben sorgt für eine direkte Rückmeldung und verbessert die Benutzerfreundlichkeit deutlich.

5.2 Herausforderungen und Lösungsansätze

Während der Entwicklung traten verschiedene Herausforderungen auf:

- Validierungslogik: Die gegenseitige Abhängigkeit der Eingabewerte erforderte eine dynamische Validierung. Dies wurde durch die Implementierung eines benutzerdefinierten Validators gelöst.
- UI-Integration: Die Eingabemaske sollte kompakt und übersichtlich bleiben, ohne die Benutzerführung zu stören. Die Entscheidung für ein Dialogfenster war hier ein effektiver Lösungsansatz.
- Berechnungslogik: Die korrekte Umsetzung der VI-Formel inklusive der Tabellenwerte und Grenzfälle erforderte sorgfältige Recherche und Tests.

Diese Herausforderungen konnten durch gezielte technische Maßnahmen und iterative Anpassungen erfolgreich gelöst werden.

5.3 Potenzial für Weiterentwicklung

Die Projektarbeit zeigt, wie sich theoretisches Wissen zur Viskositätsberechnung praxisnah in eine digitale Anwendung überführen lässt. Die aktuelle Umsetzung konzentriert sich auf die Berechnung des Viskositätsindex bei 40°C und 100°C. Für zukünftige Erweiterungen bestehen jedoch vielfältige Möglichkeiten, um die Anwendung weiterzuentwickeln und ihren Nutzen zu steigern:

- Erweiterung um weitere Temperaturbereiche: Nutzer:innen sollen künftig die Möglichkeit erhalten, den Viskositätsindex auch bei anderen Temperaturen zu berechnen, um flexibler auf unterschiedliche technische Anforderungen reagieren zu können.

- Darstellung der Viskositätsverläufe in Diagrammform: Eine visuelle Darstellung der Temperaturabhängigkeit der Viskosität würde die Ergebnisse anschaulicher machen und die Interpretation erleichtern.
- Exportfunktion für Ergebnisse: Die Möglichkeit, Berechnungen als PDF oder CSV zu exportieren, wäre besonders für die technische Dokumentation und Weiterverarbeitung hilfreich.

Diese Weiterentwicklungen würden die Anwendung nicht nur funktional erweitern, sondern auch ihren praktischen Nutzen im Bereich der Schmierstofftechnik deutlich erhöhen.

6. Fazit

Im Rahmen dieser Projektarbeit wurde ein Viskositätsindex-Rechner als Webanwendung mit Angular entwickelt und erfolgreich in die Grease App integriert. Die Umsetzung hat gezeigt, wie sich theoretische Grundlagen aus der Tribologie mit modernen Webtechnologien verbinden lassen, um ein praxisnahes und benutzerfreundliches Tool zu schaffen.

Die Arbeit bietet nicht nur technische Herausforderungen, sondern auch wertvolle Einblicke in die Gestaltung interaktiver Benutzeroberflächen. Die entwickelten Lösungsansätze haben sich als tragfähig erwiesen und bilden eine solide Grundlage für zukünftige Erweiterungen. Mit Blick auf den Ausblick zeigt sich, dass die Anwendung großes Potenzial für Weiterentwicklungen bietet.

Insgesamt stellt die Projektarbeit einen gelungenen Beitrag zur digitalen Unterstützung in der Schmierstofftechnik dar und bietet eine gute Basis für weiterführende Entwicklungen.

7. Literaturverzeichnis

Adam Freeman, Pro Angular 16, Manning, 6th Edition, 2024.

Amit Diwan, Ultimate Bootstrap for Responsive Web Design, AVA, 2024.

Aristeidis Bampakos, Learning Angular, <packt>, Fifth Edition, 2025.

Peter Himschoot, Microsoft Blazor, Building Web Applications in .NET 6 and Beyond, Apress, Third Edition, 2021.

Shyam Seshadri, Angular Up & Running, O'Reilly Media, Inc, 2024.

ASTM D2270-24, Standard Practice for Calculating Viscosity Index from Kinematic Viscosity at 40°C and 100°C.

<https://angular.dev/guide/forms/form-validation#validating-input-in-reactive-forms>, Zugriff am: 13.08.2025.

<https://www.typescriptlang.org>, Zugriff am: 04.08.2025.

8. Hilfsmittelangabe

Hilfsmittelangabe zum Einsatz von KI-basierten Werkzeugen bei der Anfertigung von wissenschaftlichen Arbeiten



Persönliche Angaben

Duong, Thi Quynh Nhi Nachname, Vorname	6889454 Matrikelnummer
Adam-Klein-Straße 83a, 90429 Nürnberg Anschrift	WWI24SEA Kurs
s241418@student.dhbw-mannheim.de E-Mail	0152 24826385 Telefon- / Handynummer

Für das Modul

Projektarbeit 1, 2.Semester Modulbezeichnung / Semester

muss ich am

25.08.2025 Datum der Frist
--

folgende Prüfungsleistung erbringen:

- ☒ Projektarbeit I ☐ Projektarbeit II
☐ Seminararbeit ☐ Bachelorarbeit

☐ Sonstige

genaue Bezeichnung der sonstigen Prüfungsleistung

Zur Verwendung KI-gestützter Werkzeuge erkläre ich in Kenntnis des Hinweisblatts "Hinweise zum Einsatz von KI-basierten Werkzeugen bei der Anfertigung wissenschaftlicher Arbeiten und die prüfungsrechtlichen Folgen ihres Einsatzes" Folgendes:

- Ich habe mich aktiv über die Leistungsfähigkeit und Beschränkungen der in meiner Arbeit eingesetzten KI-Werkzeuge informiert.
- Bei der Anfertigung der Prüfungsleistung habe ich durchgehend eigenständig und beim Einsatz KI-gestützter Werkzeuge maßgeblich steuernd gearbeitet.
- Insbesondere habe ich die Inhalte entweder aus wissenschaftlichen oder anderen zugelassenen Quellen entnommen und diese gekennzeichnet oder diese unter Anwendung wissenschaftlicher Methoden selbst entwickelt.
- Mir ist bewusst, dass ich als Autor/in der Arbeit die Verantwortung für die in ihr gemachten Angaben und Aussagen trage.
- Ich habe keine weiteren als die nachstehend von mir benannten KI-gestützten Werkzeuge zur Erstellung der Arbeit eingesetzt und diese nur in der angegebenen Art und Weise.

- Soweit ich KI-gestützte Werkzeuge zur Erstellung der Arbeit eingesetzt habe, gebe ich diese in der nachstehenden "Übersicht verwendeter Hilfsmittel" mit ihrem Produkt-namen und ihres genutzten Funktionsumfangs vollständig an; wörtliche oder sinn-gemäße Übernahmen KI-generierter Inhalte habe ich in ein separates Verzeichnis aufgenommen und im Text belegt (z. B. als Fußnote). Diese Inhalte sind als pdf-Datei in den elektronischen Beigaben hinterlegt.

Produktname(n) eingesetzter Hilfsmittel:

ChatGPT, Copilot

Genutzter Funktionsumfang im Hinblick auf

- **Themenerfassung und Strukturierung** (Aufgabenstellung oder Präzisierung, Forschungsansatz, Gliederung)

Inspiration zur Findung des Forschungsansatzes

- **Themenbearbeitung** (Darstellung/Beschreibung des Problems, Darlegung von wissenschaft-lichen Grundlagen, Schlussfolgerungen allgemein und für das konkrete Thema / Erkenntnisgewinn, Evaluierung des Textes / Feedback der KI)

Schlussfolgerung allgemein (Seite 22)

- **Quellenrecherche, -auswahl und -auswertung** (welche Quellen wurden durch das KI-Werkzeug gefunden, wie erfolgte die weitere Recherche)

- **Formale Gestaltung, insbesondere Sprache** (Welche Eingabe wurde an die KI getätigt? Textgenerierung, Textkorrektur, Paraphrasieren und Umschreiben, Übersetzen, kreatives Schreiben)

Umformulierung, Textgenerierung, grammatische Korrektur, Übersetzung (Seite 4-22)

Hinweis:

Geben Sie in der jeweiligen Rubrik die Seitenzahl in Ihrer wissenschaftlichen Arbeit an und erläutern Sie die Art und Weise sowie den Umfang der von Ihnen genutzten KI-basierten Werkzeuge.

Mannheim, 22.08.2025



Ort, Datum

Unterschrift der/des Studierenden

9. Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Feature Entwicklung: Integration der Viskositätsindex-Berechnung in die Engineering App mit Angular

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Mannheim, 22.08.2025



Thi Quynh Nhi, Duong

10. Sperrvermerk

Die vorliegende Arbeit mit dem Titel

Feature Entwicklung: Integration der Viskositätsindex-Berechnung in die Engineering App mit Angular

enthält unternehmensinterne bzw. vertrauliche Informationen der Schaeffler Technologies AG & Co. KG, ist deshalb mit einem Sperrvermerk versehen und wird ausschließlich zu Prüfungszwecken im Studiengang Wirtschaftsinformatik – Software Engineering der Dualen Hochschule Baden-Württemberg Mannheim vorgelegt.

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung der Ausbildungsstätte (Schaeffler Technologies AG & Co. KG) vorliegt.

Mannheim, 22.08.2025



Thi Quynh Nhi, Duong